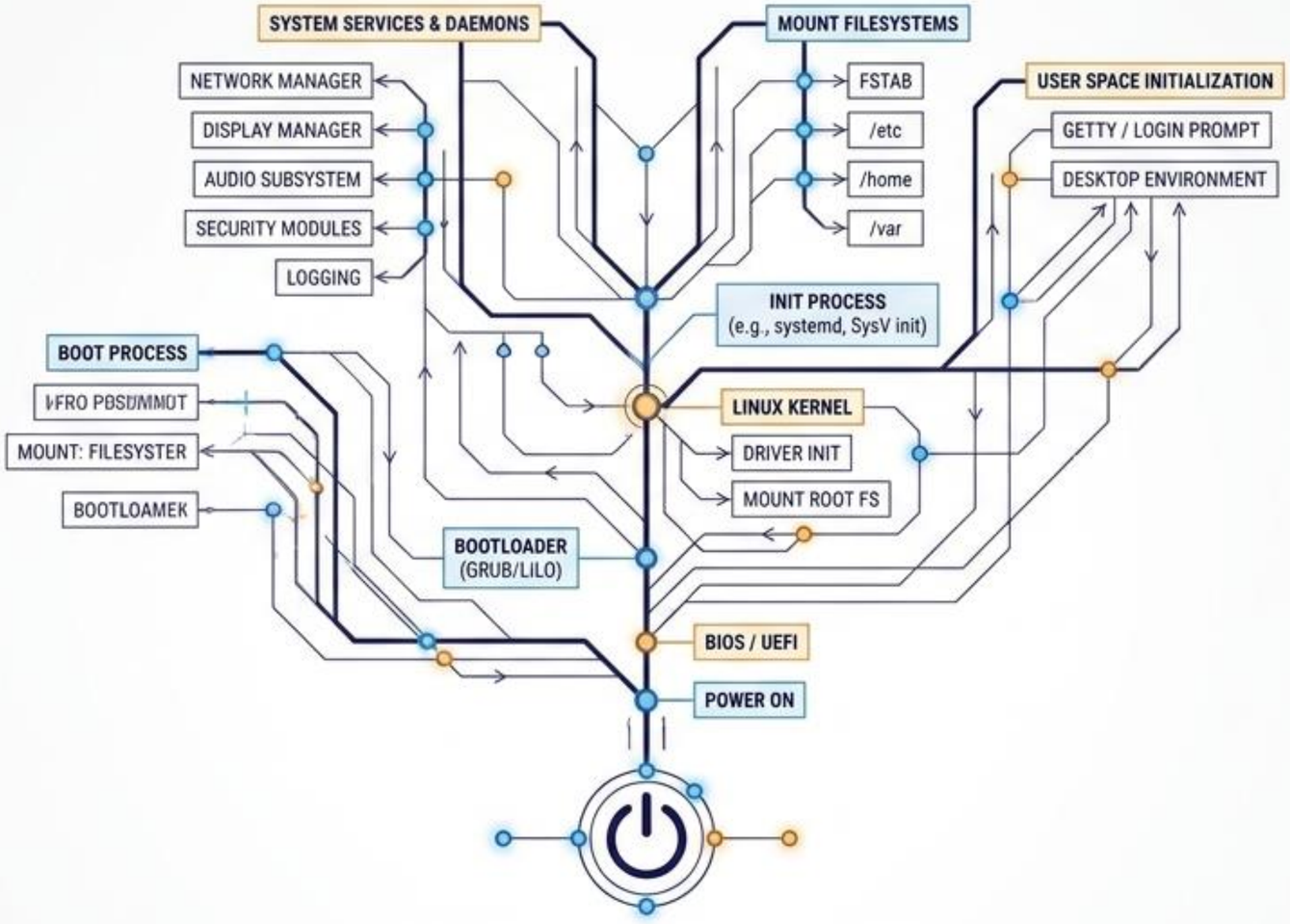
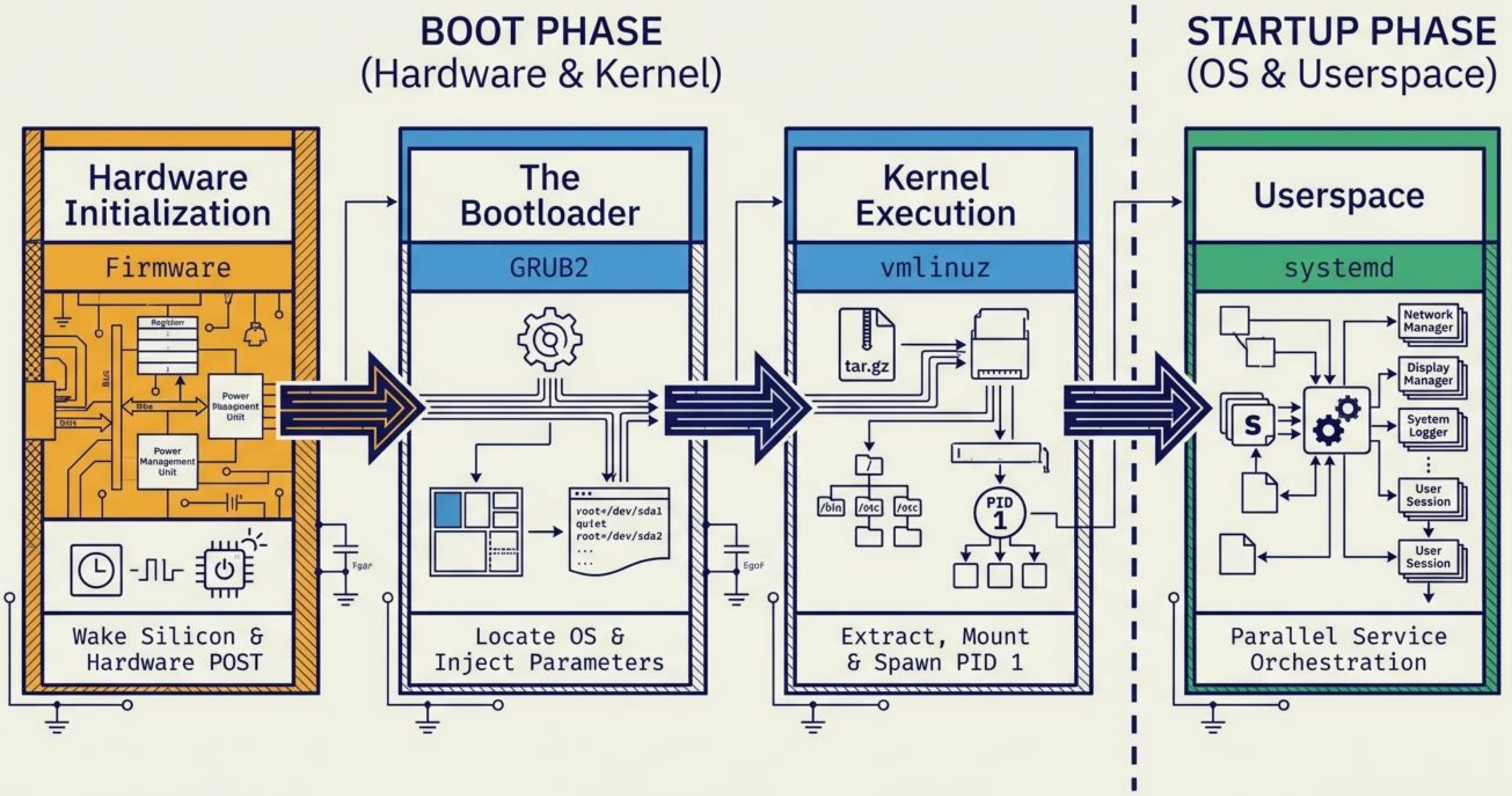
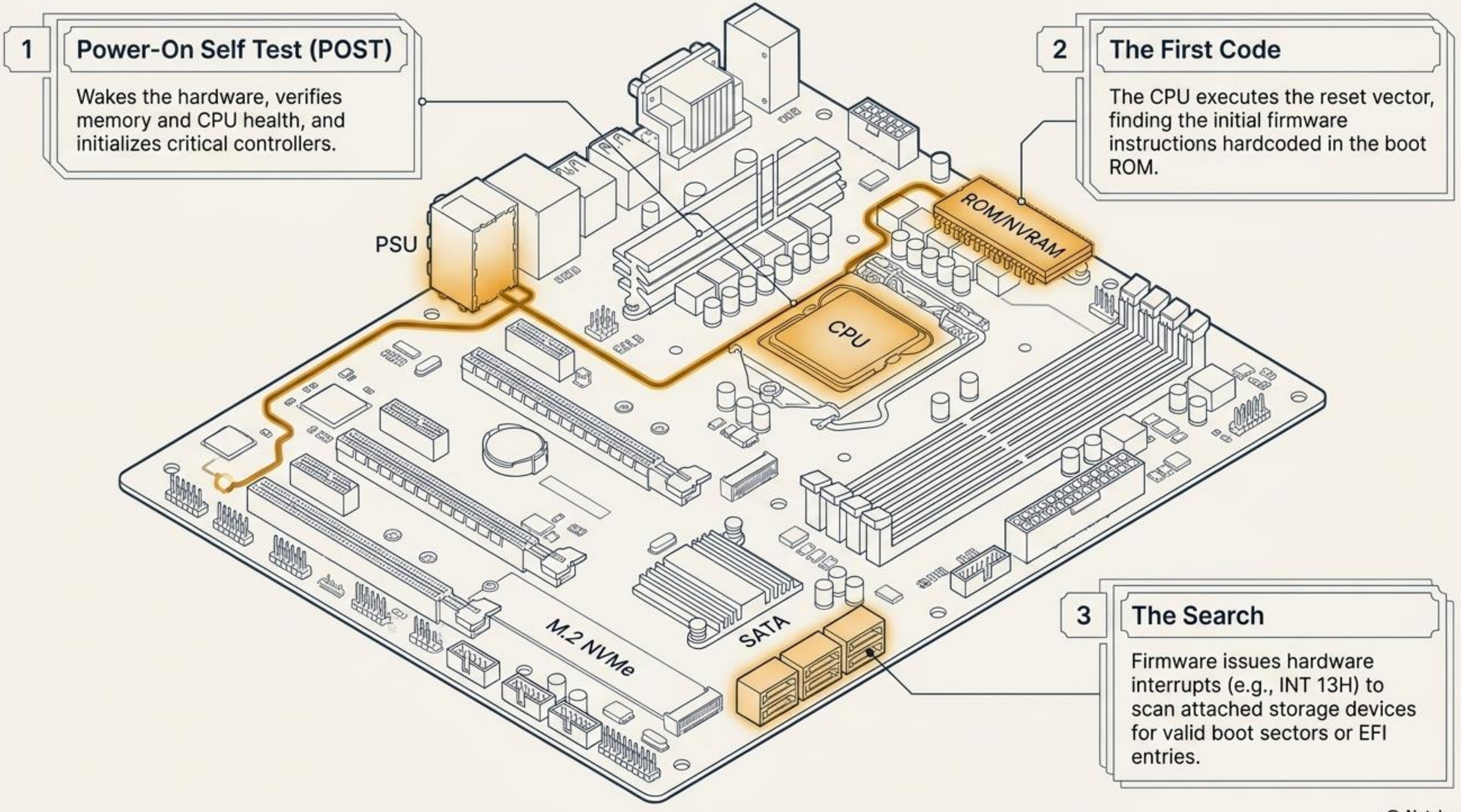


The Linux Ignition Sequence

A definitive visual guide to the boot and startup architecture


BOOT PHASE
(Hardware & Kernel)
STARTUP PHASE
(OS & Userspace)
Hardware Initialization
Firmware
Power Management Unit
Wake Silicon & Hardware POST
The Bootloader
GRUB2
root=/dev/sda1
quiet
root=/dev/sda2
Locate OS & Inject Parameters
Kernel Execution
vmlinuz
tar.gz
/bin
PID 1
Extract, Mount & Spawn PID 1
Userspace
systemd
Network Manager
Display Manager
Syetem Logger
User Session
User Session
Parallel Service Orchestration

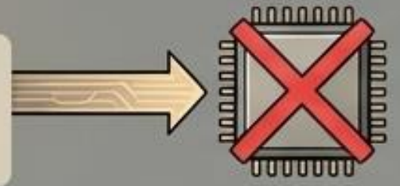

1
Power-On Self Test (POST)
Wakes the hardware, verifies memory and CPU health, and initializes critical controllers.
2
The First Code
The CPU executes the reset vector, finding the initial firmware instructions hardcoded in the boot ROM.
PSU
ROM/NVRAM
CPU
M.2 NVMe
SATA
3
The Search
Firmware issues hardware interrupts (e.g., INT 13H) to scan attached storage devices for valid boot sectors or EFI entries.

The Firmware Divide: BIOS vs. UEFI

Legacy BIOS

Architecture

16-bit architecture constraint.



Partitioning

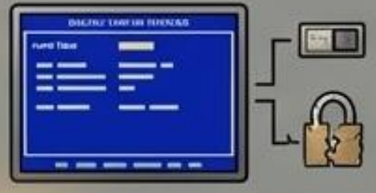
Relies on Master Boot Record (MBR). Fails on drives larger than 2TB.

Boot Mechanism

Blindly scans and executes the first 512-byte sector (Sector 0) of disks.

Security & Extensibility

Text-based interface with extremely limited hardware initialization.



Modern UEFI

Architecture

32/64-bit extensible architecture.

Partitioning

Native GUID Partition Table (GPT) support.

Boot Mechanism

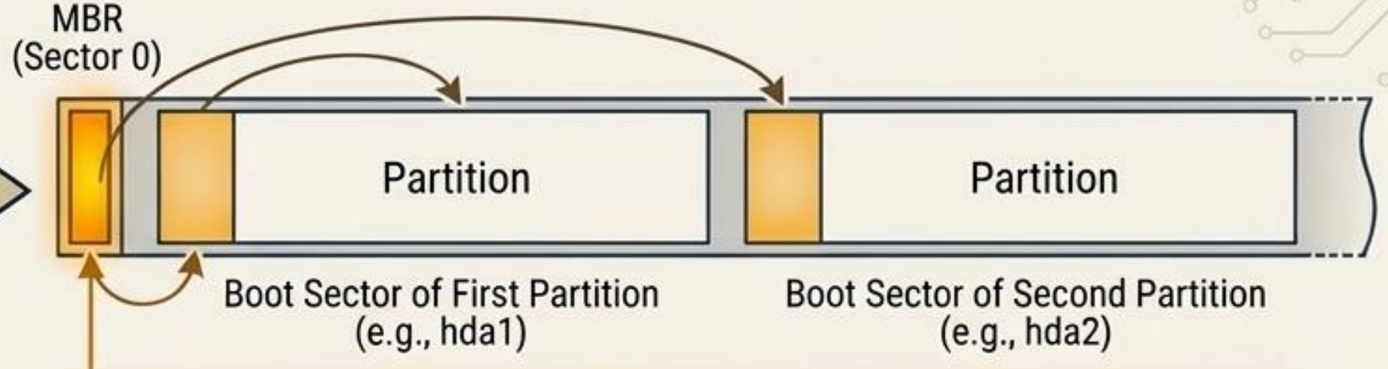
Filesystem-aware. Reads structured boot entries directly from NVRAM and executes .efi files.

Security & Extensibility

Supports modular network booting and cryptographic Secure Boot.

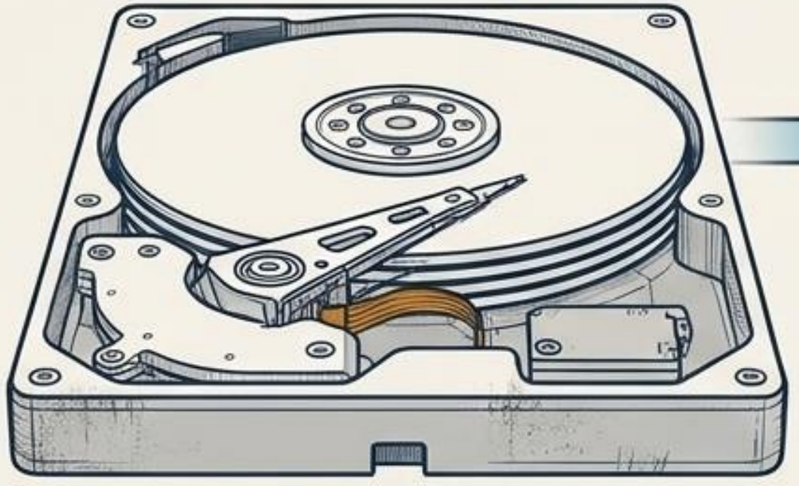
Boot Process & Storage Layouts: BIOS/MBR vs. UEFI/GPT

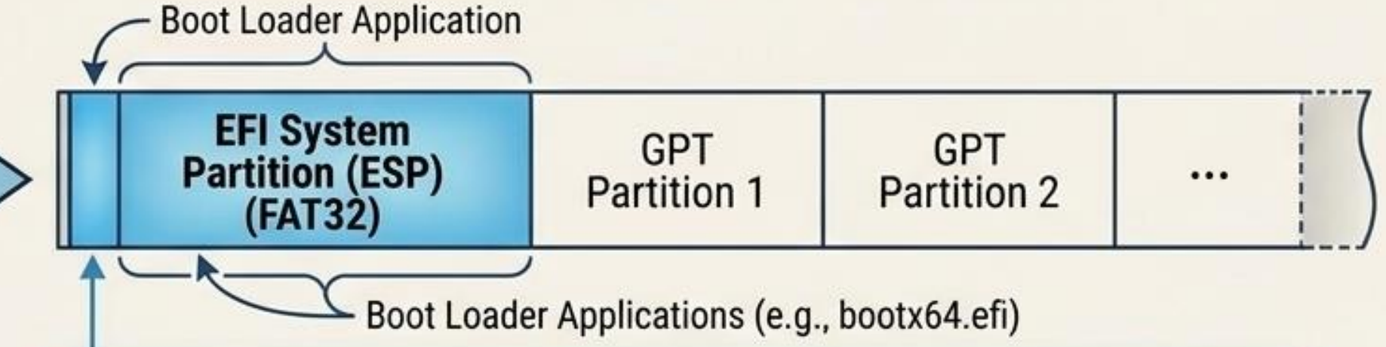
BIOS / MBR Layout



The MBR Constraint
BIOS blindly executes whatever raw code lives in Sector 0 (446 bytes). Highly brittle and heavily constrained by disk size.

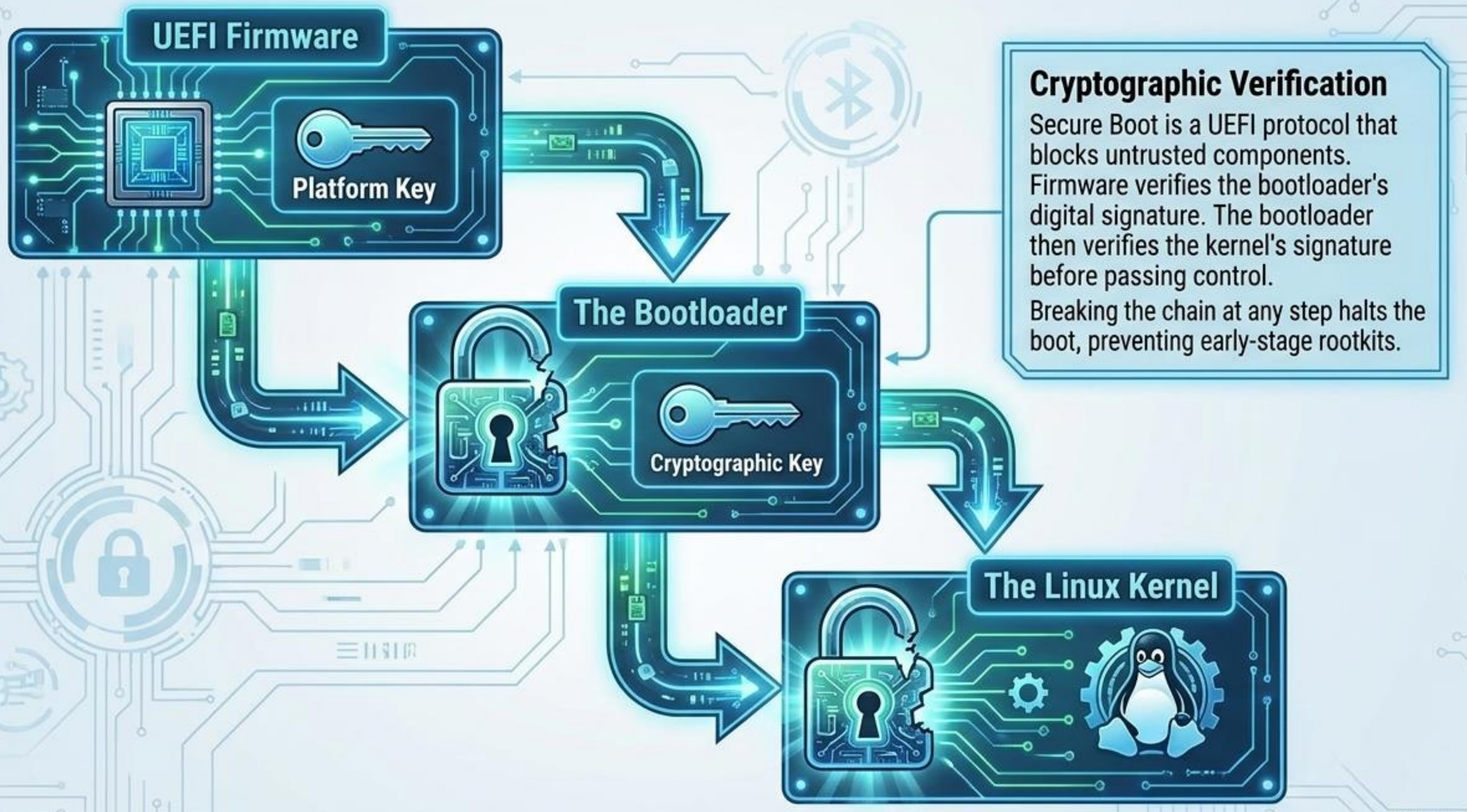
UEFI / GPT Layout





The ESP Advantage
UEFI understands modern filesystems. It mounts the ESP (FAT32) and loads structured EFI bootloader applications, eliminating reliance on raw sectors.

The Chain of Trust (Secure Boot)



Cryptographic Verification

Secure Boot is a UEFI protocol that blocks untrusted components. Firmware verifies the bootloader's digital signature. The bootloader then verifies the kernel's signature before passing control.

Breaking the chain at any step halts the boot, preventing early-stage rootkits.

The Anatomy of GRUB2

Stage 1 (boot.img)

Location: MBR (446 bytes)

A tiny, filesystem-blind loader whose sole purpose is to locate and execute Stage 1.5.

Stage 1.5 (core.img)

Location: Sector 63 Gap (~31KB)

Sits in the unused space before the first partition. Contains just enough filesystem drivers to read EXT4/XFS.

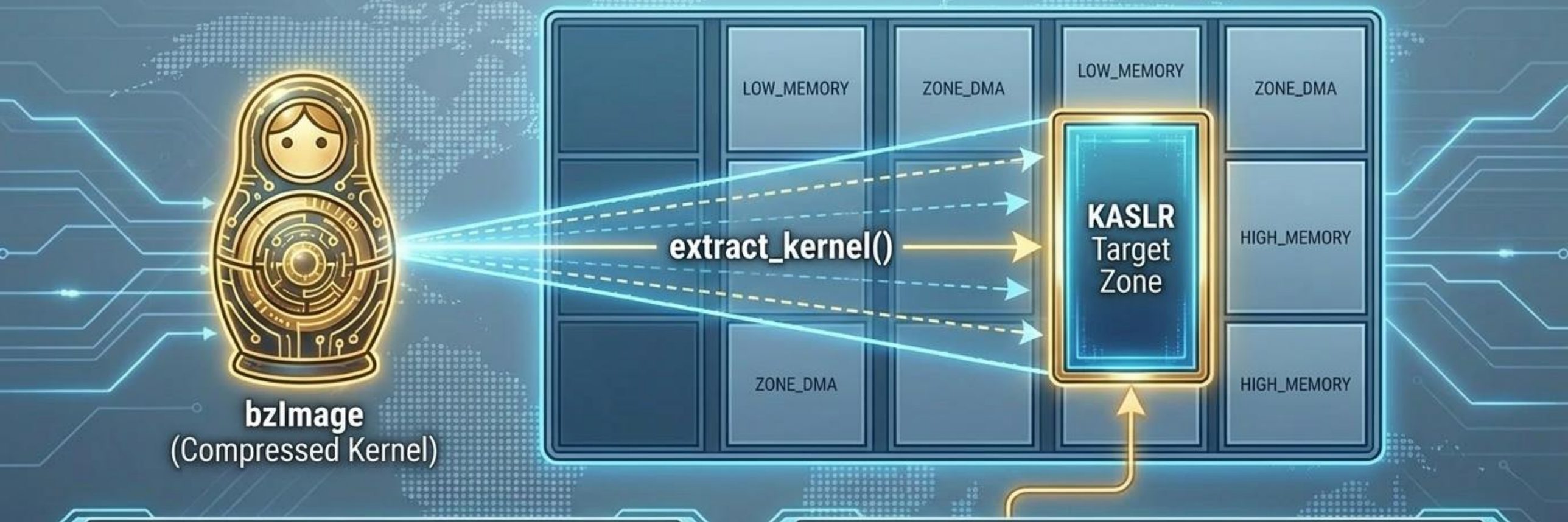
Stage 2 (/boot/grub2)

Location: Linux Filesystem

Fully filesystem-aware. Loads the UI menu, kernel modules, and the target OS into RAM.

GRUB2 bootstraps itself from a tiny, dumb sector into a fully filesystem-aware environment capable of loading complex kernels.

Kernel Execution & Decompression



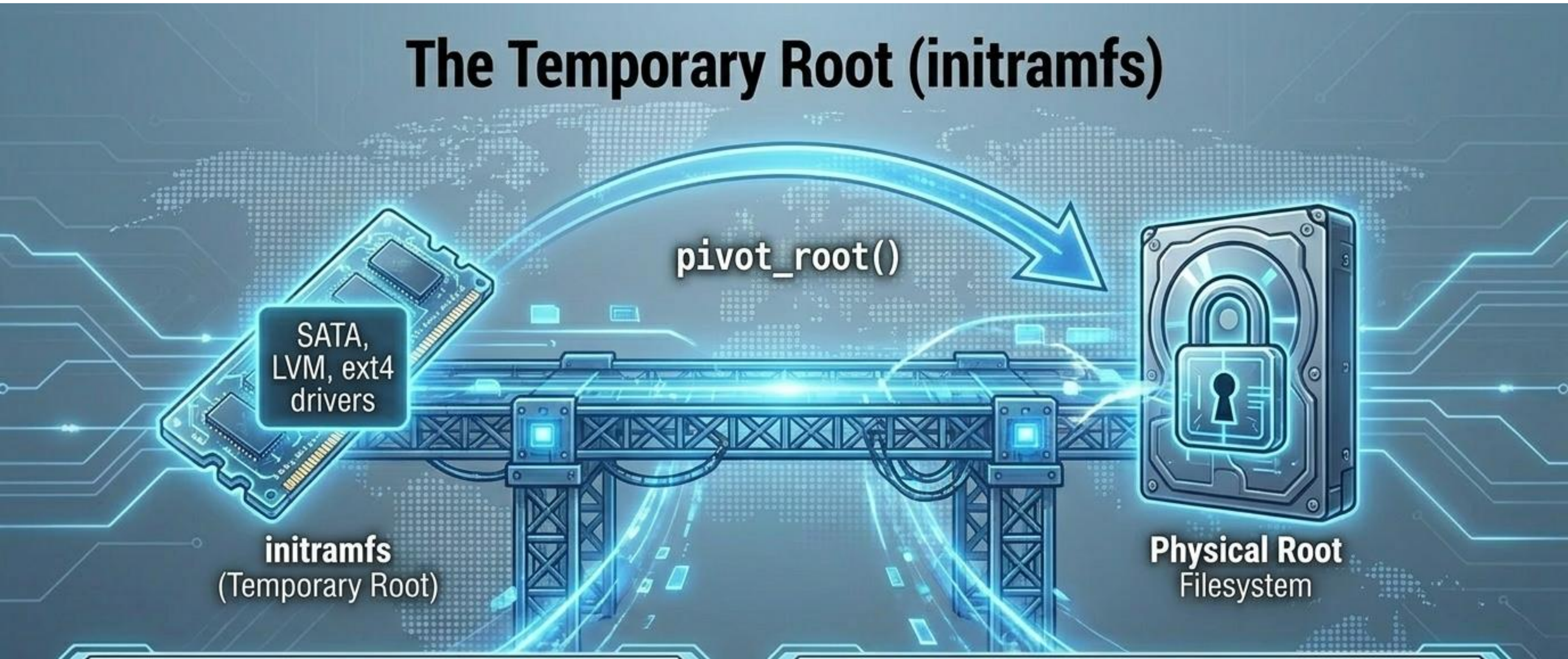
Procedural Unpacking

The kernel runs **startup_32** to clear the .bss section (uninitialized data), then invokes **extract_kernel()** to unpack itself into physical memory.

KASLR Security

Kernel Address Space Layout Randomization. The function **`choose_random_location()`** ensures the kernel never decompresses into the same physical address twice, thwarting predictable memory-based attacks.

The Temporary Root (initramfs)



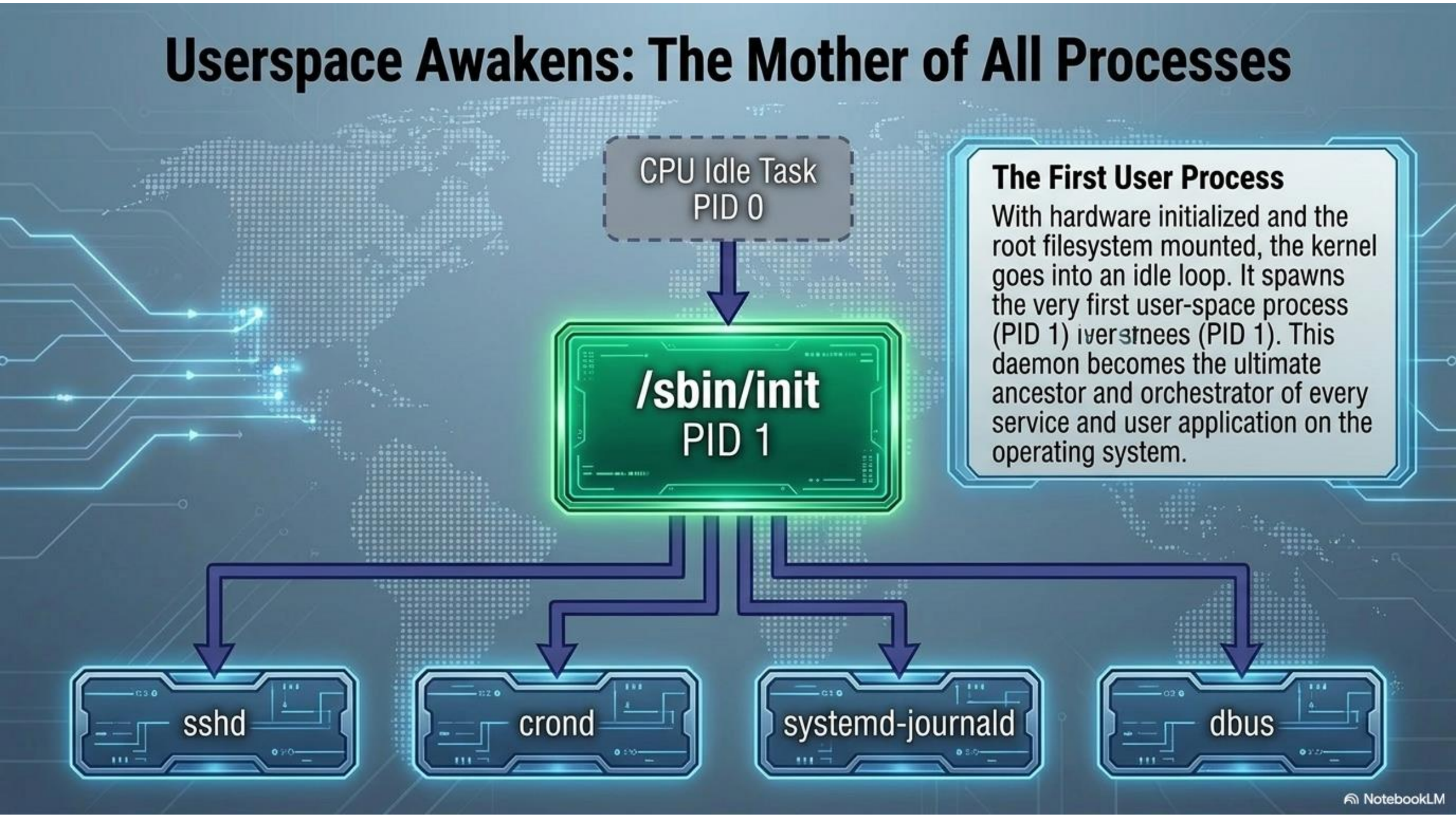
The Paradox

The kernel needs storage drivers to mount the hard drive, but those exact drivers are stored ON the hard drive.

The Solution

The bootloader pre-loads initramfs into RAM. The kernel mounts this temporary filesystem, loads the essential drivers, and uses **pivot_root()** to seamlessly swap it for the real physical root.

Userspace Awakens: The Mother of All Processes



Full Synthesis: From Power to Prompt

Firmware
(POST)

Bootloader
(GRUB2)

Kernel
(vmlinuz/initramfs)

Userspace
(systemd)

```
user@host:~$ systemd-analyze blame
   3.120s postgresql.service
   1.041s network.service
    840ms systemd-journald.service
    120ms sshd.service
```

Take Control

The Linux boot process is no longer a black box. A continuous chain of handoffs—from silicon ROM to parallelized userspace daemons—transforms inert hardware into a dynamic OS. Use `systemd-analyze` to trace, debug, and optimize your own ignition sequence.